

3.5 Şablonlar

Bir web çerçevesi olan Django'nun HTML'yi dinamik olarak oluşturmak için kullanışlı bir yönteme ihtiyacı vardır. En yaygın yaklaşım şablonlara dayanır. Bir şablon, istenen HTML çıktısının statik kısımlarının yanı sıra dinamik içeriğin nasıl ekleneceğini açıklayan bazı özel sözdizimini içerir. Şablonlarla HTML sayfaları oluşturmaya dair uygulamalı bir örnek için bkz. [Öğretici 3](#).

Bir Django projesi, bir veya birkaç şablon motoruyla yapılandırılabilir (veya şablon kullanmıyorsanız hiçbiri de kullanılmayabilir). Django, kendi şablon sistemi için yaratıcı bir şekilde Django şablon dili (DTL) olarak adlandırılan ve popüler alternatif [Jinja2](#) için yerleşik arka uçlar sunar. Diğer şablon dilleri için arka uçlar üçüncü taraflarca sağlanabilir. Ayrıca kendi özel arka ucunuzu da yazabilirsiniz, bkz. [Özel şablon arka ucu](#)

Django, arka uçtan bağımsız olarak şablonları yüklemek ve işlemek için standart bir API tanımlar. Yükleme, belirli bir tanımlayıcıya ait şablonu bulup ön işleme tabi tutmaktan, genellikle onu bellek içi bir temsil haline derlemekten ibarettir. İşleme, şablonu bağlam verileriyle birleştirip sonuçta ortaya çıkan dizgiyi döndürmek anlamına gelir.

[Django şablon dili](#), Django'nun kendi şablon sistemidir. Django 1.8'e kadar mevcut tek yerleşik seçenek buydu. Oldukça katı kurallara sahip ve birkaç tuhaflığı olsa da iyi bir şablon kütüphanesidir. Başka bir arka uç seçmek için acil bir nedeniniz yoksa, özellikle

takılabilir bir uygulama yazıyorsanız ve şablonları dağıtmayı planlıyorsanız. [django.contrib.admin](#) gibi şablonlar içeren Django'nun katkı uygulamaları DTL'yi kullanır.

Tarihsel nedenlerden ötürü, hem şablon motorları için genel destek hem de Django şablon dilinin uygulaması `django.template` ad alanında bulunur.

Uyarı

Şablon sistemi, güvenilir olmayan şablon yazarlarına karşı güvenli değildir. Örneğin, bir site, kullanıcılarının kendi şablonlarını sağlamasına izin vermemelidir; çünkü şablon yazarları, XSS saldırıları gerçekleştirebilir ve hassas bilgiler içerebilecek şablon değişkenlerinin özelliklerine erişebilir.

3.5.1 Django şablon dili

Sözdizimi

Bu bölüm hakkında

Bu bölüm, Django şablon dilinin sözdizimine genel bir bakıştır. Ayrıntılar için [dil sözdizimi referansına](#) bakın.

Bir Django şablonu, Django şablon dili kullanılarak işaretlenmiş bir metin belgesi veya Python dizesidir. Bazı yapılar şablon motoru tarafından tanınır ve yorumlanır. Bunların başlıcaları değişkenler ve etiketlerdir.

Bir şablon, bir bağlamla birlikte işlenir. İşleme sırasında değişkenler, bağlamda aranan değerleriyle değiştirilir ve etiketler yürütülür. Diğer her şey olduğu gibi çıktılanır.

Django şablon dilinin sözdizimi dört yapıdan oluşur.

Değişkenler

Bir değişken, anahtarları değerlere eşleyen sözlük benzeri bir nesne olan bağlamdan bir değer çıkarır.

Değişkenler şu şekilde `{{ ve }}` ile çevrelenir:

```
Adım {{ first_name }}. Soyadım {{ last_name }}.
```

`{'first_name': 'John', 'last_name': 'Doe'}` bağlamında, bu şablon şu şekilde görüntülenir:

```
Adım John. Soyadım Doe.
```

Sözlük araması, öznitelik araması ve liste dizini aramaları nokta notasyonu ile gerçekleştirilir:

```
{{ my_dict.key }}
{{ my_object.attribute }}
{{ my_list.0 }}
```

Bir değişken çağrılabilir bir ögeye çözümlenirse, şablon sistemi onu hiçbir argüman olmadan çağırır ve çağrılabilir öge yerine sonucunu kullanır.

Etiketler

Etiketler, görüntüleme sürecinde isteğe bağlı mantık sağlar.

Bu tanım kasıtlı olarak belirsiz bırakılmıştır. Örneğin, bir etiket içerik üretebilir, bir kontrol yapısı (ör. bir "if" ifadesi veya bir "for" döngüsü) görevi görebilir, bir veritabanından içerik alabilir veya hatta diğer şablon etiketlerine erişimi sağlayabilir.

Etiketler şu şekilde {% ve %} ile çevrelenir:

```
{% csrf_token %}
```

Çoğu etiket argüman kabul eder:

```
{% cycle 'odd' 'even' %}
```

Bazı etiketler başlangıç ve bitiş etiketleri gerektirir:

```
{% if user.is_authenticated %}Merhaba, {{ user.username }}.{% endif %}
```

Yerleşik etiketlerin bir listesi ve özel etiket yazma talimatları da mevcuttur.

Filtreler

Filtreler, değişkenlerin ve etiket argümanlarının değerlerini dönüştürür. Şu şekilde görünürler:

```
{{ django|title }}
```

{'django': 'son teslim tarihi olan mükemmeliyetçiler için web çerçevesi'} bağlamında, bu şablon şu şekilde görüntülenir:

```
Son teslim tarihi olan mükemmeliyetçiler için web çerçevesi
```

Bazı filtreler bir argüman alır:

```
{{ my_date|date:"Y-m-d" }}
```

Yerleşik filtrelerin bir listesi ve özel filtreler yazma talimatları da mevcuttur.

Yorumlar

Yorumlar şu şekilde görünür:

```
{# bu görüntülenmeyecek #}
```

`{% comment %}` etiketi, çok satırlı yorumlar oluşturur.

Bileşenler

 Bu bölüm hakkında

Bu, Django şablon dilinin API'lerine genel bir bakıştır. Ayrıntılar için [API referansına](#) bakın.

Motor

`django.template.Engine`, Django şablon sisteminin bir örneğini kapsar. Bir *Engine*'i doğrudan örneklendirmenin ana nedeni, Django şablon dilini bir Django projesi dışında kullanmaktır.

`django.template.backends.django.DjangoTemplates`, `django.template.Engine`'i Django'nun şablon arka uç API'sine uyarlayan ince bir sarmalayıcıdır.

Şablon

`django.template.Template`, derlenmiş bir şablonu temsil eder. Şablonlar, `Engine.get_template()` veya `Engine.from_string()` ile elde edilir.

Benzer şekilde, `django.template.backends.django.Template`, `django.template.Template`'i genel şablon API'sine uyarlayan ince bir sarmalayıcıdır.

Bağlam

`django.template.Context`, bağlam verilerine ek olarak bazı meta verileri de içerir. Şablonu oluşturmak için `Template.render()` işlevine aktarılır.

`django.template.RequestContext`, mevcut `HttpRequest`'i depolayan ve şablon bağlam işleyicilerini çalıştıran `Context`'in bir alt sınıfıdır.

Ortak API'de buna eşdeğer bir kavram yoktur. Bağlam verileri düz bir `dict` içinde aktarılır ve gerekirse mevcut `HttpRequest` ayrı olarak aktarılır.

Yükleyiciler

Şablon yükleyiciler, şablonları bulmak, yüklemek ve *Template* nesnelerini döndürmekten sorumludur.

Django, birkaç yerleşik şablon yükleyicisi sağlar ve özel şablon yükleyicilerini destekler.

Bağlam işleyicileri

Bağlam işleyicileri, geçerli *HttpRequest*'i bir argüman olarak alan ve görüntüleme bağlamına eklenecek verilerin bir *sözlüğünü* döndüren işlevlerdir.

Bunların temel kullanımı, her görünümde kodu tekrarlamadan tüm şablonlar tarafından paylaşılan ortak verileri bağlama eklemektir.

Django birçok yerleşik bağlam işleyicisi sağlar ve siz de kendi ek bağlam işleyicilerinizi uygulayabilirsiniz.

3.5.2 Şablon motorları için destek

Yapılandırma

Şablon motorları, *TEMPLATES* ayarı ile yapılandırılır. Bu, her motor için birer tane olmak üzere bir yapılandırma listesidir. Varsayılan değer boştur. *startproject* komutu tarafından oluşturulan *settings.py* dosyası, daha kullanışlı bir değer tanımlar:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates", "DIRS":
        [],
        "APP_DIRS": True, "OPTIONS":
        {
            #... burada bazı seçenekler ...
        },
    },
]
```

BACKEND, Django'nun şablon arka uç API'sini uygulayan bir şablon motoru sınıfına giden noktalı bir Python yoludur. Yerleşik arka uçlar *django.template.backends.django.DjangoTemplates* ve *django.template.backends.jinja2.Jinja2*'dir.

Çoğu motor şablonları dosyalardan yüklediğinden, her motorun üst düzey yapılandırması iki ortak ayar içerir:

- *DIRS*, motorun şablon kaynak dosyalarını araması gereken dizinlerin listesini arama sırasına göre tanımlar.
- *APP_DIRS*, motorun yüklü uygulamaların içinde şablon araması gerekip gerekmediğini belirtir. Her arka uç, şablonlarının depolanması gereken uygulamalar içindeki alt izin için geleneksel bir ad tanımlar

Nadiren de olsa, aynı arka ucun birkaç örneğini farklı seçeneklerle yapılandırmak mümkündür. Bu durumda, her motor için benzersiz bir *NAME* tanımlamalısınız.

OPTIONS, arka uca özgü ayarları içerir.

Kullanım

`django.template.loader` modülü, şablonları yüklemek için iki işlev tanımlar.

get_template(template_name, using=None)

Bu işlev, belirtilen ada sahip şablonu yükler ve bir `Template` nesnesi döndürür.

Dönüş değerinin tam türü, şablonu yükleyen arka uca bağlıdır. Her arka ucun kendi `Template` sınıfı vardır.

`get_template()`, başarılı olana kadar her şablon motorunu sırayla dener. Şablon bulunamazsa, *TemplateDoesNotExist* hatası verir. Şablon bulunursa ancak geçersiz sözdizimi içeriyorsa, *TemplateSyntaxError* hatası verir.

Şablonların nasıl aranacağı ve yükleneceği, her motorun arka ucuna ve yapılandırmasına bağlıdır.

Aramayı belirli bir şablon motoruyla sınırlamak *istiyorsanız*, `using` argümanına motorun `NAME`'ini geçirin.

Kısmi yükleme:

DjangoTemplates arka ucunu kullanırken, bir şablondan belirli bir parça isme göre yüklenebilir. Bu parça, önceden *partialdef* şablon etiketi kullanılarak tanımlanmış olmalıdır:

```
from django.template.loader import get_template

# Şablonun tamamını yükleyin.
template = get_template("template.html")

# Şablondan belirli bir parçayı yükleyin.
partial = get_template("template.html#partial_name")
```

Bir parçayı yüklerken, döndürülen nesne normal bir Şablon gibi davranır, ancak yalnızca parçanın içeriğini içerir.

Şablon parçalarının tanımlanması ve kullanılması hakkında daha fazla bilgi için [Şablon parçaları](#)

[bölümüne](#) bakın. *DjangoTemplates* arka ucunu kullanırken şablon parçalarını yükleme desteği

eklendi.

select_template(template_name_list, using=None)

`select_template()`, şablon adlarının bir listesini alması dışında `get_template()` ile aynıdır. Her adı sırayla dener ve var olan ilk şablonu döndürür.

Şablonun yüklenmesi başarısız olursa, `django.template` içinde tanımlanan aşağıdaki iki istisna ortaya çıkabilir:

TemplateDoesNotExist *istisnası*(msg, tried=None, backend=None, chain=None)

Bu istisna, bir şablon bulunamadığında ortaya çıkar. Hata ayıklama sayfasında [şablonu sonradan](#) doldurmak için aşağıdaki isteğe bağlı argümanları kabul eder:

backend

İstisnanın kaynaklandığı şablon arka uç örneği.

tried

Şablonu bulmaya çalışırken denenmiş kaynakların listesi. Bu, (origin, status) içeren tuple'lardan oluşan bir liste olarak biçimlendirilir; burada origin, [origin benzeri](#) bir nesne ve status, şablonun bulunamama nedenini içeren bir dizedir.

chain

Şablonu yüklemeye çalışırken ortaya çıkan ara *TemplateDoesNotExist* istisnalarının listesi. Bu, belirli bir şablonu birden fazla motorundan yüklemeye çalışan *get_template()* gibi işlevler tarafından kullanılır.

exception TemplateSyntaxError(msg)

Bu istisna, bir şablon bulunmuş ancak hatalar içeriyorsa ortaya çıkar.

get_template() ve *select_template()* tarafından döndürülen şablon nesneleri, aşağıdaki imzaya sahip bir *render()* yöntemi sağlamalıdır:

`Template.render`(context=None, request=None) Bu

şablonu belirli bir bağlamla işler.

`context` sağlanırsa, bir *dict* olmalıdır. Sağlanmazsa, motor şablonu boş bir bağlamla işler.

`request` sağlanırsa, bir *HttpRequest* olmalıdır. Ardından motor, bunu ve CSRF belirtecini şablonda kullanılabilir hale getirmelidir. Bunun nasıl gerçekleştirileceği her bir arka uca bağlıdır.

İşte arama algoritmasının bir örneği. Bu örnek için *TEMPLATES* ayarı şöyledir:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates", "DIRS": [
            "/home/html/example.com",
            "/home/html/default",
        ],
    },
    {
        "BACKEND": "django.template.backends.jinja2.Jinja2", "DIRS":
        [
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```

        "/home/html/jinja2",
    ],
},
]

```

`get_template('story_detail.html')` işlevini çağırırsanız, Django'nun sırayla arayacağı dosyalar şunlardır:

- `/home/html/example.com/story_detail.html` ('django' motoru)
- `/home/html/default/story_detail.html` ('django' motoru)
- `/home/html/jinja2/story_detail.html` ('jinja2' motoru)

`select_template(['story_253_detail.html', 'story_detail.html'])` işlevini çağırırsanız, Django şunlara bakacaktır:

- `/home/html/example.com/story_253_detail.html` ('django' motoru)
- `/home/html/default/story_253_detail.html` ('django' motoru)
- `/home/html/jinja2/story_253_detail.html` ('jinja2' motoru)
- `/home/html/example.com/story_detail.html` ('django' motoru)
- `/home/html/default/story_detail.html` ('django' motoru)
- `/home/html/jinja2/story_detail.html` ('jinja2' motoru)

Django mevcut bir şablon bulduğunda, aramayı durdurur.

 Daha fazla esneklik için **`django.template.loader.select_template()`** işlevini kullanın

Esnek şablon yükleme için *`select_template()` işlevini* kullanabilirsiniz. Örneğin, bir haber yazdınız ve bazı haberlerin özel şablonlara sahip olmasını istiyorsanız, `select_template(['story_%s_detail.html' % story.id, 'story_detail.html'])` gibi bir şey kullanın. Bu, tek tek haberler için özel bir şablon kullanmanıza ve özel şablonu olmayan haberler için yedek bir şablon kullanmanıza olanak tanır.

Şablonları, şablonları içeren her dizinin içindeki alt dizinlerde düzenlemek mümkündür ve tercih edilir. Geleneksel olarak, her Django uygulaması için bir alt dizin oluşturulur ve bu alt dizinlerin içinde gerektiğinde alt dizinler oluşturulur.

Kendi iyiliğiniz için bunu yapın. Tüm şablonları tek bir dizinin kök seviyesinde saklamak karmaşık bir durum yaratır. Bir alt dizinde bulunan bir şablonu yüklemek için, şu şekilde bir eğik çizgi

```
get_template("news/story_detail.html")
```

kullanın:

Yukarıdaki *TEMPLATES* seçeneğiyle aynı seçeneği kullanarak, bu komut aşağıdaki şablonları yüklemeye çalışacaktır:

- /home/html/example.com/news/story_detail.html ('django' motoru)
- /home/html/default/news/story_detail.html ('django' motoru)
- /home/html/jinja2/news/story_detail.html ('jinja2' motoru)

Ayrıca, şablonların yüklenmesi ve işlenmesinin tekrarlayıcı doğasını azaltmak için Django, süreci otomatikleştiren bir kısayol işlevi sunar.

render_to_string(template_name, context=None, request=None, using=None)

`render_to_string()`, *get_template()* gibi bir şablonu yükler ve hemen `render()` yöntemini çağırır. Aşağıdaki argümanları alır.

template_name

Yüklenecek ve işlenecek şablonun adı. Eğer bu bir şablon adları listesi ise, Django şablonu bulmak için *get_template()* yerine *select_template()* işlevini kullanın.

context

Oluşturma için şablonun bağlamı olarak kullanılacak bir [sözlük](#).

request

Şablonun işlenmesi sırasında kullanılabilecek isteğe bağlı bir *HttpRequest*.

using

İsteğe bağlı bir şablon motoru *ADI*. Şablon araması bu motorla sınırlandırılacaktır.

Kullanım örneği:

```
from django.template.loader import render_to_string

rendered = render_to_string("my_template.html", {"foo": "bar"})
```

Ayrıca, *render_to_string()* işlevini çağırarak ve sonucu bir görünümünden döndürülmeye uygun bir *HttpResponse*'a aktaran *render()* kısayoluna da bakın.

Son olarak, yapılandırılmış motorları doğrudan kullanabilirsiniz:

engines

Şablon motorları `django.template.engines` içinde mevcuttur:

```
from django.template import engines

django_engine = engines["django"]
template = django_engine.from_string("Merhaba {{ name }}!")
```

Arama anahtarı — bu örnekte 'django' — motorun *ADI*'dir.

Yerleşik arka uçlar

class DjangoTemplates

Bir Django şablon motorunu yapılandırmak için *BACKEND*'i 'django.template.backends.django.DjangoTemplates' olarak ayarlayın.

APP_DIRS True olduğunda, DjangoTemplates motorları, kurulu uygulamaların templates alt dizininde şablonları arar. Bu genel ad, geriye dönük uyumluluk için korunmuştur.

DjangoTemplates motorları aşağıdaki *OPTIONS* parametrelerini kabul eder:

- 'autoescape': HTML otomatik kaçış karakterlerinin etkinleştirilip etkinleştirilmeyeceğini kontrol eden bir boole değeri. Varsayılan değeri True'dur.

Uyarı

Yalnızca HTML olmayan şablonları işliyorsanız False olarak ayarlayın!

- 'context_processors': bir şablon bir istekle işlendiğinde bağlamı doldurmak için kullanılan çağrılabilir işlevlere giden noktalı Python yollarının listesi. Bu çağrılabilir işlevler, argüman olarak bir istek nesnesini alır ve bağlama eklenecek öğelerin bir *sözlüğü*nü döndürür.

Varsayılan olarak boş bir listedir.

Daha fazla bilgi için *RequestContext*'e bakın.

- 'debug': şablon hata ayıklama modunu açan/kapanan bir boole değeri. True ise, şık hata sayfası şablon işlenmesi sırasında ortaya çıkan herhangi bir istisna için ayrıntılı bir rapor görüntüler. Bu rapor, uygun satırın vurgulandığı şablonun ilgili parçasını içerir.

Varsayılan değer, *DEBUG* ayarının değeridir.

- 'loaders': şablon yükleyici sınıflarına giden noktalı Python yollarının bir listesidir. Her Loader sınıfı, belirli bir kaynaktan şablonları nasıl içe aktaracağını bilir. İsteğe bağlı olarak, bir dizgenin yerine bir tuple kullanılabilir. Tuple'daki ilk öge Loader sınıf adı olmalı ve sonraki öğeler başlatma sırasında Loader'a aktarılır.

Varsayılan değer, *DIRS* ve *APP_DIRS* değerlerine bağlıdır.

Ayrıntılar için *Yükleyici türleri bölümüne* bakın.

- 'string_if_invalid': şablon sisteminin geçersiz (ör. yazım hatası olan) değişkenler için kullanması gereken çıktı.

Varsayılan değer boş bir dizedir.

Ayrıntılar için *Geçersiz değişkenlerin işlenmesi bölümüne* bakın.

- 'file_charset': diskteki şablon dosyalarını okumak için kullanılan karakter kümesi.

Varsayılan değer 'utf-8'dir.

- 'libraries': Şablon motoruna kaydedilecek şablon etiket modüllerinin etiketleri ve noktalı Python yollarını içeren bir sözlük. Bu, yeni kütüphaneler eklemek veya mevcut kütüphaneler için alternatif etiketler sağlamak için kullanılabilir. Örneğin:

```
OPTIONS = {
    "libraries": {
        "myapp_tags": "path.to.myapp.tags",
        "admin_urls": "django.contrib.admin.template_tags.admin_urls",
    },
}
```

Kütüphaneler, ilgili sözlük anahtarını `{% load %}` etiketine aktararak yüklenebilir.

- 'builtins': Yerleşiklere eklenecek şablon etiketi modüllerinin noktalı Python yollarının listesi. Örneğin:

```
OPTIONS = {
    "builtins": ["myapp.builtins"],
}
```

Yerleşik kütüphanelerdeki etiketler ve filtreler, önce `{% load %}` etiketini çağırmadan kullanılabilir.

class Jinja2

Jinja2'nin kurulu olması gerekir:

```
$ python -m pip install Jinja2
```

Jinja2 motorunu yapılandırmak için *BACKEND*'i 'django.template.backends.jinja2.Jinja2' olarak ayarlayın.

APP_DIRS True olduğunda, Jinja2 motorları yüklü uygulamaların `jinja2` alt dizininde şablonları arar.

OPTIONS içindeki en önemli giriş 'environment'dır. Bu, Jinja2 ortamını döndüren bir çağrılabilir işlevin noktalı Python yoludur. Varsayılan değeri 'jinja2.Environment'dır. Django bu çağrılabilir işlevi çağırır ve diğer seçenekleri anahtar kelime argümanları olarak geçirir. Ayrıca, Django birkaç seçenek için Jinja2'den farklı varsayılan değerler ekler:

- 'autoescape': True
- 'loader': *DIRS* ve *APP_DIRS* için yapılandırılmış bir yükleyici
- 'auto_reload': settings.DEBUG
- 'undefined': settings.DEBUG ise DebugUndefined, aksi takdirde

Undefined Jinja2 motorları ayrıca aşağıdaki *OPTIONS* değerlerini de kabul eder:

- 'context_processors': bir şablon bir istekle işlendiğinde bağlamı doldurmak için kullanılan çağrılabilir işlevlere giden noktalı Python yollarının listesi. Bu çağrılabilir işlevler, argüman olarak bir istek nesnesini alır

ve bağlama eklenecek öğelerin bir **sözlüğünü** döndürür.

Varsayılan olarak boş bir listedir.

i Jinja2 şablonlarıyla bağlam işlemcileri kullanılması önerilmez.

Bağlam işlemcileri, Django şablonlarında kullanışlıdır çünkü Django şablonları, argümanlarla işlev çağırmaı desteklemez. Jinja2'de bu sınırlama bulunmadığından, bağlam işlemcisi olarak kullanacağınız işlevi, aşağıda açıklandığı gibi `jinja2.Environment` kullanarak şablonun erişebileceği global değişkenlere yerleştirmeniz önerilir. Ardından bu işlevi şablonda çağırabilirsiniz:

```
{{ function(request) }}
```

Bazı Django şablon bağlam işleyicileri sabit bir değer döndürür. Jinja2 şablonları için, sabitleri doğrudan `jinja2.Environment` içine ekleyebileceğinizden bu dolaylı katman gerekli değildir.

Jinja2 için bağlam işleyicileri eklemenin orijinal kullanım senaryosu şunları içeriyordu:

- İsteğe bağlı olarak maliyetli bir hesaplama yapmak.
- Sonucun her şablonda gerekli olması.
- Sonucun her şablonda birden çok kez kullanılması.

Tüm bu koşullar karşılanmadıkça, şablona bir işlev aktarmak Jinja2'nin tasarımına daha uygundur.

Varsayılan yapılandırma kasıtlı olarak en basit haliyle tutulmuştur. Bir şablon bir istekle işlenirse (örneğin `render()` kullanıldığında), Jinja2 arka ucu `request`, `csrf_input` ve `csrf_token` küresel değişkenlerini bağlama ekler. Bunun dışında, bu arka uç Django tarzı bir ortam oluşturmaz. Django filtreleri ve etiketleri hakkında bilgi sahibi değildir. Django'ya özgü API'leri kullanmak için, bunları ortama yapılandırmanız gerekir.

Örneğin, şu içeriğe sahip bir `myproject/jinja2.py` dosyası oluşturabilirsiniz:

```
from django.template.tags.static import static
from django.urls import reverse

from jinja2 import Environment

def environment(**options):
    env = Environment(**options)
    env.globals.update(
        {
            "static": static,
            "url": reverse,
        }
    )
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
)
return env
```

ve 'environment' seçeneğini 'myproject.jinja2.environment' olarak ayarlayın. Ardından Jinja2 şablonlarında aşağıdaki yapıları kullanabilirsiniz:

```


<a href="{{ url('admin:index') }}">Yönetim</a>
```

Etiket ve filtre kavramları hem Django şablon dilinde hem de Jinja2'de mevcuttur, ancak farklı şekillerde kullanılırlar. Jinja2, şablonlarda çağrılabilir işlevlere argümanların aktarılmasını desteklediğinden, Django şablonlarında şablon etiketi veya filtre gerektiren birçok özellik, yukarıdaki örnekte gösterildiği gibi Jinja2 şablonlarında bir işlev çağrılarak gerçekleştirilebilir. Jinja2'nin küresel ad alanı, şablon bağlam işleyicilerine olan ihtiyacı ortadan kaldırır. Django şablon dilinde Jinja2 testlerine eşdeğer bir özellik yoktur.

3.6 Sınıf tabanlı görünüm

Bir görünüm, bir istek alan ve bir yanıt döndüren bir çağrılabilir öğedir. Bu, sadece bir işlevden daha fazlası olabilir ve Django, görünüm olarak kullanılacak bazı sınıfların örneklerini sunar. Bunlar, miras ve karışımları kullanarak görünümlerinizi yapılandırmanıza ve kodu yeniden kullanmanıza olanak tanır. Daha sonra ele alacağımız bazı genel görevler için genel görünüm de vardır, ancak kullanım durumunuza uygun, yeniden kullanılabilir görünümlerden oluşan kendi yapınızı tasarlamak isteyebilirsiniz. Tüm ayrıntılar için [sınıf tabanlı görünüm referans belgelerine](#) bakın.